AWS SAA-C03 Study Notes: Amazon S3 Advanced Features

1. S3 Lifecycle Rules

Automate moving objects between storage classes or deleting them to optimize costs.

Types of Actions

- **Transition Actions:** Move objects to a cheaper storage class (e.g., Standard → Standard-IA → Glacier).
- **Expiration Actions:** Delete objects after a specified period (e.g., delete log files after 365 days).
- **Incomplete Multipart Uploads:** Configure a rule to clean up incomplete uploads to save storage costs.

Rules & Constraints

- **128KB Minimum:** Objects must be larger than 128KB to transition to Standard-IA or One Zone-IA (otherwise, you pay for 128KB).
- **30-Day Minimum:** Objects usually must sit in Standard for 30 days before transitioning to IA (Standard-IA/One Zone-IA).

S3 Analytics - Storage Class Analysis

- **Exam Tip:** "How do I know *when* to move objects to Standard-IA?"
- Use **S3 Analytics** first. It analyzes storage access patterns and generates a report suggesting optimal lifecycle rules.
- It specifically helps identify candidates for **Standard-IA** or **One Zone-IA**.

2. S3 Event Notifications

Automated reactions to events happening inside your S3 bucket.

- **Triggers:** `ObjectCreated`, `ObjectRemoved`, `ObjectRestore`, `Replication`.
- **Destinations (Targets):**
  1. **SNS** (Simple Notification Service) - Send alerts/emails.
  2. **SQS** (Simple Queue Service) - Decouple and process events later.
  3. **Lambda** - Trigger code immediately (e.g., generate a thumbnail when an image is uploaded).
  4. **EventBridge** - Complex filtering and routing to many other AWS services.
- **Versioning:** Ensure notification is enabled for specific object versions if versioning is on.

3. S3 Performance Optimization

S3 scales automatically, but specific features help with high throughput.

Baseline Performance

- **3,500** PUT/COPY/POST/DELETE requests per second (per prefix).
- **5,500** GET/HEAD requests per second (per prefix).
- *Note:* You can scale higher by using different "prefixes" (folder paths) to parallelize requests.

High-Performance Features

1. **Multipart Upload:**
   - Uploads a file in parts in parallel.
   - **Recommended** for files > 100 MB.
   - **Mandatory** for files > 5 GB.
2. **S3 Transfer Acceleration:**
   - Uses AWS **Edge Locations** to speed up uploads over the public internet.
   - Data enters the Edge Location and travels over the fast AWS internal network to the bucket.
3. **Byte-Range Fetches:**
   - Download a file in parts in parallel (speeds up downloads).
   - Can also be used to retrieve only a specific part of a file (e.g., the header of a video).
4. **S3 Select:**
   - Uses **SQL expressions** to retrieve only a specific subset of data from an object.
   - **Filters data server-side**, which drastically reduces the amount of data transferred and lowers costs.
   - **Supports** CSV, JSON, and Parquet files (even if compressed with GZIP/BZIP2).

4. S3 Batch Operations

Managed solution to perform a single action on billions of objects across buckets.

Use Cases

- **Encrypt un-encrypted objects:** Retroactively apply SSE-KMS or SSE-S3 to millions of existing legacy objects.
- **Copy objects:** Bulk copy between buckets (e.g., for disaster recovery or migration).
- **Restore from Glacier:** Bulk restore archived data.
- **Replace Tag sets:** Update cost-allocation tags.
- **Invoke Lambda:** Run custom logic on every object in the list.

The Workflow (S3 Inventory + Athena)

For Batch Operations to work, it needs a **list** of objects.

1. **S3 Inventory:** Enable this to generate a daily CSV/ORC/Parquet report listing all objects and their metadata (encryption status, size, etc.).
2. **Athena:** Query the S3 Inventory file using SQL to **filter** the list (e.g., *"Select all objects where encryption status is 'Server-Side Encryption: None'"*).
3. **Batch Job:** Feed this filtered list into S3 Batch Operations to perform the action (e.g., Encrypt).

5. S3 Storage Lens

"Single pane of glass" visibility into object storage usage and activity organization-wide.

Free vs. Paid Metrics

There are two tiers of metrics in Storage Lens:

1. **Free Metrics:** Default.
   - **Summary:** Total storage, object count, average object size.
   - **Cost Efficiency:** Identify incomplete multipart uploads, old versions, uncompressed files.
   - **Data Protection:** Bucket encryption status, replication status.
2. **Advanced Metrics (Paid):** Adds "Activity" and "Status Codes" insights.
   - **Activity Metrics:** Tracks request counts (GET, PUT, HEAD), bytes downloaded/uploaded.
   - **Detailed Status Code Metrics:** Tracks HTTP errors like **403 Forbidden** (Access Denied), **404 Not Found**, and **5xx** Server Errors.
   - **Access Management Metrics:** Tracks Object Ownership settings and encryption counts.
   - **Performance Metrics:** Tracks transfer acceleration usage and request latency.
   - **Event Metrics:** Tracks S3 Event Notification delivery rates

6. S3 Requester Pays

Shift the cost of data transfer and requests from the bucket owner to the requester.

Key Concepts

- **Who Pays What?**
  - **Requester:** Pays for the **Data Transfer** costs and the **Request** costs (GET, PUT, LIST).
  - **Bucket Owner:** Pays for the **Storage** costs.

Exam Constraints

1. **Authentication Required:** You **cannot** use Requester Pays with anonymous (public) users. The requester *must* be an authenticated AWS user so AWS knows who to bill.
2. **Header Requirement:** The requester must explicitly agree to pay by including a specific header in their API call.
   - **Header:** `x-amz-request-payer`
   - **CLI Flag:** `--request-payer requester`
3. **Failure Scenario:** If a user tries to access the bucket *without* this header, the request will fail with a **403 Access Denied** error, even if they have full permissions.